NexoEventos

Lista de Endpoints

Ingeniería de Datos – Universidad del Rosario
Entregable 1

Encabezados y Respuestas Globales

- **Encabezado:** Todas las rutas aceptan `X-Usuario`.
- **Códigos HTTP:** `201` Creado, `204` Borrado, `404` No existe, `409` Conflicto/Regla BD, `422` Datos inválidos, `503` Base no responde.

1. Catálogo de Endpoints

Método	Ruta	Descripción / Qué hace
GET, POST	`/clientes/`	Listar o buscar (q, tipo) y registrar
GET	`/clientes/{id}`	Consultar cliente
GET	`/clientes/{id}/eventos`	Eventos de un cliente
GET, POST	`/staff/`	Listar (area, q) y registrar con jefe
GET	`/staff/{id}`	Consultar empleado
GET	`/salones/`	Catálogo de salones
GET	`/salones/disponibles?...`	Salones disponibles (A5)
PATCH	`/salones/{id}`	Habilitar o deshabilitar
GET	`/salones/{id}/servicios/`	Catálogo de servicios
PATCH	`/servicios/{id}`	Cambiar precio o disponibilidad
GET, POST	`/eventos/`	Listar con filtros y crear
GET	`/eventos/{id}`	Detalle con servicios y resumen de asistencia
PATCH	`/eventos/{id}`	Reprogramar: salón, horario, aforo, coordinador
PATCH	`/eventos/{id}/estado`	Confirmar, finalizar o cancelar
GET, POST	`/eventos/{id}/servicios`	Ver y agregar servicios
PUT, DELETE	`/eventos/{id}/servicios/{id}`	Cambiar cantidad o quitar servicio
GET	`/eventos/{id}/cotizacion`	Desglose salón + servicios = total
GET	`/eventos/{id}/auditoria`	Historial de cambios
GET, POST	`/eventos/{id}/inscripciones`	Inscritos e inscribir por documento
POST	`/eventos/{id}/checkin`	Registrar llegada por documento
GET, POST	`/asistentes/`	Listar o buscar y registrar
GET	`/asistentes/{id}`	Consultar asistente
GET	`/reportes/organigrama`	Reporte organigrama (B1)
GET	`/reportes/escalamiento/{id}`	Reporte de escalamiento (B2)
GET	`/reportes/ranking-clientes`	Reporte ranking clientes (B3)
GET	`/reportes/ocupacion-salones`	Reporte ocupación salones (B4)
GET	`/reportes/tablero`	Datos del tablero de inicio

2. Ejemplos de Petición y Respuesta

Salones disponibles (A5)

```
GET /salones/disponibles?inicio=2026-11-18T09:00&fin=2026-11-18T11:00&aforo=50
200 [
  {"salon_id": 2, "nombre": "Salon Heliconia", "tamano": "Pequeno", "capacidad": 60,
     "precio_hora": 320000.0, "costo_estimado": 640000.0},
  {"salon_id": 4, "nombre": "Salon Ceiba", "tamano": "Mediano", "capacidad": 200, "
     precio_hora": 800000.0, "costo_estimado": 1600000.0},
  {"salon_id": 5, "nombre": "Gran Salon Condor", "tamano": "Grande", "capacidad":
     600, "precio_hora": 1800000.0, "costo_estimado": 3600000.0}
]
```

Crear evento que se cruza (A1)

```
POST /eventos/
{"cliente_id": 2, "salon_id": 3, "nombre": "Taller de Economia Circular", "tipo": "
    Seminario", "inicio": "2026-11-18T12:30", "fin": "2026-11-18T14:00", "
    aforo_esperado": 50}

409 {"detail": "DISPONIBILIDAD: El salon \"Salon Guayacan\" esta reservado para \"
    Jornada Cientifica Solvida\" (2026-11-18 08:00 a 2026-11-18 12:00). Se requiere 1
     hora de montaje entre eventos.", "regla": "DISPONIBILIDAD", "codigo_sql": "P0001
    ", "sugerencia": "Consulte los salones disponibles para ese horario."}
```

Finalizar (A4)

```
PATCH /eventos/6/estado (Header: X-Usuario: farango)
{"estado": "Finalizado"}
200 {"id": 6, "nombre": "Feria Empresarial del Caribe", "estado": "Finalizado", "
    valor_total": 28050000.0, "tasa_asistencia": 75.0}
```

3. Integración API – Base de Datos

`db.commit()` dispara los triggers; inmediatamente después `db.refresh(obj)` trae los campos que calculó la base (`valor_total`, `tasa_asistencia`). Si un trigger lanza `RAISE EXCEPTION`, SQLAlchemy la entrega como `DBAPIError`; `app/errores.py` lee `exc.orig.pgcode` y `exc.orig.diag` (mensaje, hint, restricción) y arma la respuesta sin trazas de Python. Todo el SQL manual usa `text()` con parámetros (`:id`), nunca concatenación, y la cadena de conexión vive en `env`.